第一页：全栈仿真——让网络等待回到推理服务

我们的目标，是解释MoE推理中的跨服务器通信为什么会拖慢生成，并比较在相同资源下，怎样的部署和通信方式更合适。研究重点是网络：token被送到哪些专家，这些专家放在哪些GPU上，通信又经过哪个NIC和交换机出口，最后怎样影响本轮以及下一轮Decode。

这里说的“全栈”，是把请求调度、计算与通信任务、网络传输和服务反馈连接起来。它并不意味着已经模拟了GPU每个时钟周期，也不意味着已经覆盖训练、Prefill和所有推理模型。

这条链路分成五步。第一步，由LLMServingSim根据请求、batch和KV状态选出本轮任务，并读取相应请求在当前层、当前Decode位置的专家选择。第二步，结合专家放置，把这些选择变成明确的计算、发送和接收任务，算清每个GPU对之间的通信字节。第三步，用Chakra保存依赖，由ASTRA在前驱完成后执行任务。第四步，将跨机消息交给独立的原生RDMA ns-3后端，在同一持续网络里模拟发送、路由、交换机排队和流控；同机通信使用共享同一时钟的简化织构。第五步，将实际完成反馈上去：接收数据就绪后，后续计算才能继续；整轮完成后，前端才能推进请求、决定下一轮组批。

因此，我们输入的是带依赖的任务，而不是预先写死开始和结束时间的数据包表。多个推理副本共享同一网络状态，可以互相争用资源。网络慢了，会改变后面的计算和服务进度。

目前，四份真实R1路由已经完成4卡、两台服务器、两个Pod、两层两步的接入核验；完整结构Qwen基准中，128卡单EP和256、512卡多EP4副本已有完成证据。这几套实验的任务并不相同，不能拿完成时间直接计算扩卡加速比。千卡两次均在600秒预算处超时，还没有完整验收；计算也仍使用声明成本，GPU标签不能当作A100或H100的实机校准结果。

第二页：拟增加细节——从流量热点走向网络机制

接下来，我们不打算平均地扩展所有功能，而是围绕一个问题增加必要细节：真实专家偏好和物理部署，在什么条件下会形成跨机网络的关键等待，怎样在不增加总资源的情况下减少这些等待。

第一类细节是路由与通信组织。现在已经能保留真实专家选择，并生成GPU对通信矩阵；接下来需要扩展来源明确的请求样本，保留热点持续性和专家共选关系，同时研究目的GPU或节点去重，以及“机内汇聚、跨机交换、机内分发”的分层通信。必须把打包、转发、合并和缓冲代价一起计入，不能只计算减少的跨机字节。热门专家也不必然形成网络拥塞：如果它就在请求本卡，跨机流量可能反而更少。

第二类细节是物理连接与分层容量。我们已有Clos、Agg、rail和平面的参数化输入；下一步需要补清多GPU共享NIC、GPU与NIC亲和、跨rail中继和实际分流机制，分别检查ToR、Agg和Core出口。比较单平面、多平面或不同拓扑时，要固定总带宽并记录交换端口等资源，避免把多加硬件带来的收益算成算法收益。现有小例中，双平面并没有自动更快，这正说明需要看实际消息和路径。

第三类细节是协议等待与跨层归因。现有网络已经有数据就绪、ACK完成和队列摘要，但摘要不能说明一段等待如何持续。我们想为选定端口增加有界的队列时序，并关联PFC暂停、ECN拥塞标记、拥塞控制反馈，以及计算的真实前驱。最终要判断慢在端侧发送、交换机排队还是确认反馈，哪些等待真正推迟了服务，而不是把所有等待简单相加。

第四类细节是校准与规模效率。需要在匹配的A100或H100、软件和消息形状下采集计算及通信数据，再检查留出负载中的预测误差；同时拆分构图、ASTRA执行、逐包事件和写出成本，在保留网络争用语义的前提下改善仿真效率，最后重新验收千卡。

验证顺序是：先在小图比较真实路由、保留频率的顺序打乱和均匀控制，再做等容量专家放置及固定带宽对照；确认网络机制后，选择专家放置或分层通信中的一种方法，完成硬件校准和代表性规模验证。最后希望形成的成果，是有明确适用边界的网络方法、可复核论文和可重复使用的实验基准。